

Križci in krožci

TicTacToe

Lara Ničetin, 63210225, VIN 2023/24

Projekt je narejen na plošči STM32H750B-DK in z uporabo grafičnega vmesnika TouchGFX (framework) (<https://www.st.com/en/development-tools/touchgfxdesigner.html>). Implementiran je algoritem Minimax za izračun najboljše poteze, ko uporabnik igra proti računalniku.

Video: [tictactoe_video.mp4](#)

Github repozitorij: <https://github.com/laranicetin/tictactoe>

OneNote zvezek: [VIN-VSP 202324 zvezek](#)

Opis projekta:

Ker rada združujem lepe barve in elemente z vrsticami kode, sem se odločila za implementacijo igre križcev in krožcev na STM32H7 plošči., a mi ni dovolj, da je igra le funkcionalna. Želela sem si, da tudi dobro izgleda in ima igravec celotno izkušnjo. Odločila sem se za uporabo vmesnika TouchGFX in za temo vesolja, ki smo jo uporabili že pri skupinskem projektu, ko smo predstavljali GPS. Igra tako popelje igralca v vesolje, sam uporabniški vmesnik je pa zelo preprost.

Programska oprema:

Za izdelavo projekta sem potrebovala sledeče programe: CubeIDE (1.16.0), TouchGFX Designer (4.23.1), CubeMX (6.12.0) in CubeProgrammer (v2.17.0).

Igra:

Igra križcev in krožcev na mreži 3x3. Igra je za dva igralca, kjer si izmenjujeta potezi. Rezultati se shranjujejo v lestvici.

Ozadja in grafični elementi:

Ker imam ozadje iz oblikovanja, sem se lotila dopolnjevanja tudi estetskega dela projekta. Ozadja so narejena v programu Adobe Illustrator. Najprej sem znotraj TouchGFX Designer-ja skicirala, kje bodo postavljeni gumbi. Nato sem v Illustratorju postavila oblike za gumbe na ista mesta in izdelala grafiko okoli nje.

Končen izgled v TouchGFX Designer, začetni zaslon.



Osnoven prototip:

Osnoven prototip aplikacije je sestavljen iz 11 zaslonov (v aplikaciji "screen"): 1. Začetni zaslon (Play), 2. izbira igre (1 / 2 player), 3. igra z 1 igralcem, 4. igra z 2 igralcema, 6 zaslonov za konec igre in zaslon z gumbi. Zadnji zaslon ni viden in njegov pomen je opisan spodaj pri problemih.

Generiranje kode:

Da sem lahko naredila funkcionalno igro, je bilo potrebno veliko urejanja kode. TouchGFX je precej omejen kar se tiče funkcij, zato jo urejamo v generiranih datotekah. Generira se en kup datotek, mene pa so najbolj zanimala le tiste, ki vsebujejo elemente na zaslonu in njihove akcije, shranjene kot imezaslonaViewBase.cpp. Koda je napisana v C++.

Pisanje kode za igro:

Najprej sem se lotila lažjega dela – igra za 2 igralca. Kar je bilo potrebno narediti: igralec klikne na zaslon, izriše se znakec, na vrsti je drugi igralec, klikne na zaslon, izriše se drug znakec. To je narejeno s kodo:

```
if (&src == &topMiddle)
{
    if (fields[0][1] == 0) {
        if (player == 1) {
            topMiddleKrizec.setVisible(true);
            player = 2;
```

```

        fields[0][1] = 1;
    } else {
        topMiddleKrozec.setVisible(true);
        player = 1;
        fields[0][1] = 2;
    }
}
}

```

Prva vrstica je samo vir klika. Najprej preverim, če je polje prosto. Če je, pogledamo kateri igralec je na vrsti. Igralec 1 je križec in igralec 2 je krožec. Ko odigrata potezo, je na vrsti drugi igralec (temu služi spremenljivka "player"). Polje fields se posodobi (več o tem pozneje).

To vse je možno napisati že znotraj programa TouchGFX. Da se program sploh lahko požene, sem morala dodati še nekaj kode.

```

int fields[3][3] = {
    { 0, 0, 0,},
    { 0, 0, 0,},
    { 0, 0, 0,},
};

```

```

void resetFields() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            fields[i][j] = 0;
        }
    }
}

```

```

int game = 1;

```

```

void checkWin() {
    if ((fields[0][0] == fields[0][1] && fields[0][1] == fields[0][2] && fields[0][0] != 0) || // prva
vrsta
        (fields[1][0] == fields[1][1] && fields[1][1] == fields[1][2] && fields[1][0] != 0) || //
druga vrsta
        (fields[2][0] == fields[2][1] && fields[2][1] == fields[2][2] && fields[2][0] != 0) || //
tretja vrsta
        (fields[0][0] == fields[1][0] && fields[1][0] == fields[2][0] && fields[0][0] != 0) || // prvi
stolpec

```

```

        (fields[0][1] == fields[1][1] && fields[1][1] == fields[2][1] && fields[0][1] != 0) || //
drugi stolpec

        (fields[0][2] == fields[1][2] && fields[1][2] == fields[2][2] && fields[0][2] != 0) || //
tretji stolpec

        (fields[0][0] == fields[1][1] && fields[1][1] == fields[2][2] && fields[0][0] != 0) || // prva
diagonala

        (fields[0][2] == fields[1][1] && fields[1][1] == fields[2][0] && fields[0][2] != 0)) { //
druga diagonala

            game = 0; // game over;

        } else if (fields[0][0] != 0 && fields[0][1] != 0 && fields[0][2] != 0 &&

                fields[1][0] != 0 && fields[1][1] != 0 && fields[1][2] != 0 &&

                fields[2][0] != 0 && fields[2][1] != 0 && fields[2][2] != 0) {

            game = 2; // draw

        } else {

            game = 1; // game continues

        }

    }

int player = 1;

```

Manjka samo še konec igre. Ko ugotovimo, kakše je izid igre, se pokaže primeren zaslon: zmaga za križec, zmaga za krožec ali izenačeno. Sprememba zaslonov je zapisana s spodnjo kodo.

```

checkWin();

if (game == 0) {

    if (player == 2) {

        application().gotokrizecWin2ScreenNoTransition();

        resetFields();

    } else {

        application().gotokrozecWin2ScreenNoTransition();

        resetFields();

    }

} else if (game == 2) {

    application().gotoizenaceno2ScreenNoTransition();

    resetFields();

}

Koda za 2 igralca izgleda tako.

// modified code

// -----

```

```
// 0 - noben, 1 - igralec 1, 2- igralec 2
```

```
int fields[3][3] = {  
    { 0, 0, 0},  
    { 0, 0, 0},  
    { 0, 0, 0},  
};
```

```
void resetFields() {  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 3; j++) {  
            fields[i][j] = 0;  
        }  
    }  
}
```

```
int game = 1;
```

```
void checkWin() {  
    if ((fields[0][0] == fields[0][1] && fields[0][1] == fields[0][2] && fields[0][0] != 0) || // prva  
vrsta  
        (fields[1][0] == fields[1][1] && fields[1][1] == fields[1][2] && fields[1][0] != 0) || //  
druga vrsta  
        (fields[2][0] == fields[2][1] && fields[2][1] == fields[2][2] && fields[2][0] != 0) || //  
tretja vrsta  
        (fields[0][0] == fields[1][0] && fields[1][0] == fields[2][0] && fields[0][0] != 0) || // prvi  
stolpec  
        (fields[0][1] == fields[1][1] && fields[1][1] == fields[2][1] && fields[0][1] != 0) || //  
drugi stolpec  
        (fields[0][2] == fields[1][2] && fields[1][2] == fields[2][2] && fields[0][2] != 0) || //  
tretji stolpec  
        (fields[0][0] == fields[1][1] && fields[1][1] == fields[2][2] && fields[0][0] != 0) || // prva  
diagonala  
        (fields[0][2] == fields[1][1] && fields[1][1] == fields[2][0] && fields[0][2] != 0)) { //  
druga diagonala  
        game = 0; // game over;  
    } else if (fields[0][0] != 0 && fields[0][1] != 0 && fields[0][2] != 0 &&  
        fields[1][0] != 0 && fields[1][1] != 0 && fields[1][2] != 0 &&  
        fields[2][0] != 0 && fields[2][1] != 0 && fields[2][2] != 0) {  
        game = 2; // draw  
    } else {  
        game = 1; // game continues  
    }  
}
```

```

    }
}

int player = 1;

void game_2ViewBase::flexButtonCallbackHandler(const touchgfx::AbstractButtonContainer& src)
{
    if (&src == &topLeft)
    {
        if (fields[0][0] == 0) {
            if (player == 1) {
                topLeftKrizec.setVisible(true);
                player = 2;
                fields[0][0] = 1;
            } else {
                topLeftKrozec.setVisible(true);
                player = 1;
                fields[0][0] = 2;
            }
        }
    }
    if (&src == &topMiddle)
    {
        if (fields[0][1] == 0) {
            if (player == 1) {
                topMiddleKrizec.setVisible(true);
                player = 2;
                fields[0][1] = 1;
            } else {
                topMiddleKrozec.setVisible(true);
                player = 1;
                fields[0][1] = 2;
            }
        }
    }
    if (&src == &topRight)
    {
        if (fields[0][2] == 0) {
            if (player == 1) {

```

```

        topRightKrizec.setVisible(true);
        player = 2;
        fields[0][2] = 1;
    } else {
        topRightKrozec.setVisible(true);
        player = 1;
        fields[0][2] = 2;
    }
}

}

if (&src == &middleLeft)
{
    if (fields[1][0] == 0) {
        if (player == 1) {
            middleLeftKrizec.setVisible(true);
            player = 2;
            fields[1][0] = 1;
        } else {
            middleLeftKrozec.setVisible(true);
            player = 1;
            fields[1][0] = 2;
        }
    }
}

}

if (&src == &middleMiddle)
{
    if (fields[1][1] == 0) {
        if (player == 1) {
            middleMiddleKrizec.setVisible(true);
            player = 2;
            fields[1][1] = 1;
        } else {
            middleMiddleKrozec.setVisible(true);
            player = 1;
            fields[1][1] = 2;
        }
    }
}
}

```

```

}

if (&src == &middleRight)
{
    if (fields[1][2] == 0) {
        if (player == 1) {
            middleRightKrizec.setVisible(true);
            player = 2;
            fields[1][2] = 1;
        } else {
            middleRightKrozec.setVisible(true);
            player = 1;
            fields[1][2] = 2;
        }
    }
}

if (&src == &bottomLeft)
{
    if (fields[2][0] == 0) {
        if (player == 1) {
            bottomLeftKrizec.setVisible(true);
            player = 2;
            fields[2][0] = 1;
        } else {
            bottomLeftKrozec.setVisible(true);
            player = 1;
            fields[2][0] = 2;
        }
    }
}

if (&src == &bottomMiddle)
{
    if (fields[2][1] == 0) {
        if (player == 1) {
            bottomMiddleKrizec.setVisible(true);
            player = 2;
            fields[2][1] = 1;
        } else {
            bottomMiddleKrozec.setVisible(true);
            player = 1;
        }
    }
}

```



```

        fields[2][1] = 2;
    }
}
if (&src == &bottomRight)
{
    if (fields[2][2] == 0) {
        if (player == 1) {
            bottomRightKrizec.setVisible(true);
            player = 2;
            fields[2][2] = 1;
        } else {
            bottomRightKrozec.setVisible(true);
            player = 1;
            fields[2][2] = 2;
        }
    }
}

// check if over
// players so zamenjani, ker se v zankah zamenja turn
checkWin();
if (game == 0) {
    if (player == 2) {
        application().gotokrizecWin2ScreenNoTransition();
        resetFields();
    } else {
        application().gotokrozecWin2ScreenNoTransition();
        resetFields();
    }
}

} else if (game == 2) {
    application().gotoizenaceno2ScreenNoTransition();
    resetFields();
}
}

void game_2ViewBase::buttonCallbackHandler(const touchgfx::AbstractButton& src)
{

```

```

    if (&src == &back)
    {
        //back

        //When back clicked change screen to player

        //Go to player with no screen transition
        application().gotoplayerScreenNoTransition();
        resetFields();

        game = 1;
    }

    if (&src == &restart)
    {
        application().gotogame_2ScreenBlockTransition();
        resetFields();

        game = 1;
    }
}

```

Igra za 1 igralca je bolj komplicirana. Morala sem biti pozorna na to, kaj se dogaja ko je pritisnjen en del ozadja, saj bo tukaj le 1 akcija povzročila ostale.

Osnova igre za 1 igralca je enaka kot za 2, z nekaj manjšimi razlikami. Najprej sem se morala odločiti za algoritem. Pri igri križcev in krožcev se za ugotavljanje najbolj optimalne poteze uporablja algoritem Minimax. Ta deluje tako, da izračuna naslednje možne poteze in ugotovi, kateri se njemu najbolj izplača. Tukaj me je skrbelo, da bi slabo optimiziran algoritem povzročil težave pri izvajanju (a do tega ni prišlo).

Na spletu je že precej algoritmov, ki vsak zase trdi, da je najbolj optimiziran, najhitrejši ali najboljši. Jaz sem si svojega izposodila od Geeks for geeks (<https://www.geeksforgeeks.org/finding-optimal-move-in-tic-tac-toe-using-minimax-algorithm-in-game-theory/>).

```
// modified code
```

```
// -----
```

```
// 0 - noben, 1 - igralec 1, 2- igralec 2
```

```

int fields1[3][3] = {
    { 0, 0, 0,},
    { 0, 0, 0,},
    { 0, 0, 0,},
};

```

```

void resetFields1() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            fields1[i][j] = 0;
        }
    }
}

int game1 = 1;
int player1 = 1;
int fields_copy[3][3];

void checkWin1() {
    if ((fields1[0][0] == fields1[0][1] && fields1[0][1] == fields1[0][2] && fields1[0][0] != 0) || //
        prva vrsta
        (fields1[1][0] == fields1[1][1] && fields1[1][1] == fields1[1][2] && fields1[1][0] != 0) || //
        druga vrsta
        (fields1[2][0] == fields1[2][1] && fields1[2][1] == fields1[2][2] && fields1[2][0] != 0) || //
        tretja vrsta
        (fields1[0][0] == fields1[1][0] && fields1[1][0] == fields1[2][0] && fields1[0][0] != 0) || //
        prvi stolpec
        (fields1[0][1] == fields1[1][1] && fields1[1][1] == fields1[2][1] && fields1[0][1] != 0) || //
        drugi stolpec
        (fields1[0][2] == fields1[1][2] && fields1[1][2] == fields1[2][2] && fields1[0][2] != 0) || //
        tretji stolpec
        (fields1[0][0] == fields1[1][1] && fields1[1][1] == fields1[2][2] && fields1[0][0] != 0) || //
        prva diagonalna
        (fields1[0][2] == fields1[1][1] && fields1[1][1] == fields1[2][0] && fields1[0][2] != 0)) { //
        druga diagonalna

        game1 = 0; // game over;

    } else if (fields1[0][0] != 0 && fields1[0][1] != 0 && fields1[0][2] != 0 &&
        fields1[1][0] != 0 && fields1[1][1] != 0 && fields1[1][2] != 0 &&
        fields1[2][0] != 0 && fields1[2][1] != 0 && fields1[2][2] != 0) {

        game1 = 2; // draw

    } else {

        game1 = 1; // game continues

    }
}

void copyFields() {
    for (int i = 0; i < 3; i++) {

```

```

        for (int j = 0; j < 3; j++) {
            fields_copy[i][j] = fields1[i][j];
        }
    }
}

```

// MINIMAX

// algoritem za iskanje najboljše poteze

struct Move

```

{
    int row, col;
};

```

// This function returns true if there are moves left to play

```

bool isMovesLeft(int board[3][3]) {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (board[i][j] == 0) // 0 means blank
                return true;
    return false;
}

```

// This function evaluates the board for a win, returning +10 for a win by the player (1), -10 for a win by the opponent (2), and 0 otherwise.

```

int evaluate(int b[3][3]) {
    // Checking rows for victory
    for (int row = 0; row < 3; row++) {
        if (b[row][0] == b[row][1] && b[row][1] == b[row][2]) {
            if (b[row][0] == 1) // Player 1 (cross)
                return +10;
            else if (b[row][0] == 2) // Player 2 (circle)
                return -10;
        }
    }
}

```

// Checking columns for victory

```

for (int col = 0; col < 3; col++) {
    if (b[0][col] == b[1][col] && b[1][col] == b[2][col]) {

```

```

        if (b[0][col] == 1)
            return +10;
        else if (b[0][col] == 2)
            return -10;
    }
}

// Checking diagonals for victory
if (b[0][0] == b[1][1] && b[1][1] == b[2][2]) {
    if (b[0][0] == 1)
        return +10;
    else if (b[0][0] == 2)
        return -10;
}

if (b[0][2] == b[1][1] && b[1][1] == b[2][0]) {
    if (b[0][2] == 1)
        return +10;
    else if (b[0][2] == 2)
        return -10;
}

// No winner
return 0;
}

int minimax(int board[3][3], int depth, bool isMax) {
    int score = evaluate(board);

    // If the game is over, return the evaluated score
    if (score == 10)
        return score;
    if (score == -10)
        return score;

    // If there are no moves left and no winner, it's a tie
    if (!isMovesLeft(board))
        return 0;

```

```

// Maximizer's move (Player 1 - cross)
if (isMax) {
    int best = -1000;

    // Traverse all cells
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == 0) { // Check if the cell is empty
                board[i][j] = 1; // Make the move

                best = max(best, minimax(board, depth + 1, false));

                board[i][j] = 0; // Undo the move
            }
        }
    }
    return best;
}

// Minimizer's move (Player 2 - circle)
else {
    int best = 1000;

    // Traverse all cells
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == 0) { // Check if the cell is empty
                board[i][j] = 2; // Make the move

                best = min(best, minimax(board, depth + 1, true));

                board[i][j] = 0; // Undo the move
            }
        }
    }
    return best;
}
}

Move findBestMove(int board[3][3]) {

```

```

    int bestVal = -1000;

    Move bestMove;

    bestMove.row = -1;

    bestMove.col = -1;

    // Traverse all cells, evaluate the minimax function for all empty cells
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == 0) { // Check if the cell is empty
                board[i][j] = 1; // Make the move

                int moveVal = minimax(board, 0, false); // Evaluate the move

                board[i][j] = 0; // Undo the move

                if (moveVal > bestVal) {
                    bestMove.row = i;
                    bestMove.col = j;
                    bestVal = moveVal;
                }
            }
        }
    }

    return bestMove;
}

/*void makeComputerMove() {
    Move bestMove = findBestMove(fields1); // Use fields directly

    if (bestMove.row != -1 && bestMove.col != -1) {
        fields1[bestMove.row][bestMove.col] = 2; // Computer (circle) moves
        // Update the UI accordingly
    }
}*/

void game_1ViewBase::flexButtonCallbackHandler(const touchgfx::AbstractButtonContainer& src)
{
    // player 1

```

```

if (player1 == 1) {
    if (&src == &topLeft)
    {
        if (fields1[0][0] == 0) {
            topLeftKrizec.setVisible(true);
            topLeftKrizec.invalidate();
            player1 = 2;
            fields1[0][0] = 1;
        }
    }
    if (&src == &topMiddle)
    {
        if (fields1[0][1] == 0) {
            topMiddleKrizec.setVisible(true);
            topMiddleKrizec.invalidate();
            player1 = 2;
            fields1[0][1] = 1;
        }
    }
    if (&src == &topRight)
    {
        if (fields1[0][2] == 0) {
            topRightKrizec.setVisible(true);
            topRightKrizec.invalidate();
            player1 = 2;
            fields1[0][2] = 1;
        }
    }
    if (&src == &middleLeft)
    {
        if (fields1[1][0] == 0) {
            middleLeftKrizec.setVisible(true);
            middleLeftKrizec.invalidate();
            player1 = 2;
            fields1[1][0] = 1;
        }
    }
}

```



```

if (&src == &middleMiddle)
{
    if (fields1[1][1] == 0) {
        middleMiddleKrizec.setVisible(true);
        middleMiddleKrizec.invalidate();
        player1 = 2;
        fields1[1][1] = 1;
    }
}

if (&src == &middleRight)
{
    if (fields1[1][2] == 0) {
        middleRightKrizec.setVisible(true);
        middleRightKrizec.invalidate();
        player1 = 2;
        fields1[1][2] = 1;
    }
}

if (&src == &bottomLeft)
{
    if (fields1[2][0] == 0) {
        bottomLeftKrizec.setVisible(true);
        bottomLeftKrizec.invalidate();
        player1 = 2;
        fields1[2][0] = 1;
    }
}

if (&src == &bottomMiddle)
{
    if (fields1[2][1] == 0) {
        bottomMiddleKrizec.setVisible(true);
        bottomMiddleKrizec.invalidate();
        player1 = 2;
        fields1[2][1] = 1;
    }
}

if (&src == &bottomRight)
{
    if (fields1[2][2] == 0) {

```

```

        bottomRightKrizec.setVisible(true);
        bottomRightKrizec.invalidate();
        player1 = 2;
        fields1[2][2] = 1;
    }
}
}

checkWin1();

if (game1 == 0) {
    if (player1 == 2) {
        application().gotokrizecWin1ScreenNoTransition();
        resetFields1();
    } else {
        application().gotokrozecWin1ScreenNoTransition();
        resetFields1();
    }
}

} else if (game1 == 2) {
    application().gotoizenaceno1ScreenNoTransition();
    resetFields1();
}

copyFields();
Move bestMove = findBestMove(fields_copy);

// player 2 - computer

if (player1 == 2) {
    if (bestMove.row == 0 && bestMove.col == 0)
    {
        topLeftKrozec.setVisible(true);
        topLeftKrozec.invalidate();
        player1 = 1;
        fields1[0][0] = 2;
    }

    if (bestMove.row == 0 && bestMove.col == 1)

```

```
{
    topMiddleKrozec.setVisible(true);
    topMiddleKrozec.invalidate();
    player1 = 1;
    fields1[0][1] = 2;
}
if (bestMove.row == 0 && bestMove.col == 2)
{
    topRightKrozec.setVisible(true);
    topRightKrozec.invalidate();
    player1 = 1;
    fields1[0][2] = 2;

}
if (bestMove.row == 1 && bestMove.col == 0)
{
    middleLeftKrozec.setVisible(true);
    middleLeftKrozec.invalidate();
    player1 = 1;
    fields1[1][0] = 2;

}
if (bestMove.row == 1 && bestMove.col == 1)
{
    middleMiddleKrozec.setVisible(true);
    middleMiddleKrozec.invalidate();
    player1 = 1;
    fields1[1][1] = 2;
}
if (bestMove.row == 1 && bestMove.col == 2)
{
    middleRightKrozec.setVisible(true);
    middleRightKrozec.invalidate();
    player1 = 1;
    fields1[1][2] = 2;
}
if (bestMove.row == 2 && bestMove.col == 0)
{
    bottomLeftKrozec.setVisible(true);
```

```

        bottomLeftKrozec.invalidate();

        player1 = 1;
        fields1[2][0] = 2;
    }
    if (bestMove.row == 2 && bestMove.col == 1)
    {
        bottomMiddleKrozec.setVisible(true);
        bottomMiddleKrozec.invalidate();
        player1 = 1;
        fields1[2][1] = 2;
    }
    if (bestMove.row == 2 && bestMove.col == 2)
    {
        bottomRightKrozec.setVisible(true);
        bottomRightKrozec.invalidate();
        player1 = 1;
        fields1[2][2] = 2;
    }
}

checkWin1();

if (game1 == 0) {
    if (player1 == 2) {
        application().gotokrizecWin1ScreenNoTransition();
        resetFields1();
    } else {
        application().gotokrozecWin1ScreenNoTransition();
        resetFields1();
    }
}

} else if (game1 == 2) {
    application().gotoizenaceno1ScreenNoTransition();
    resetFields1();
}

}

void game_1ViewBase::buttonCallbackHandler(const touchgfx::AbstractButton& src)

```

```

{
    if (&src == &back)
    {
        //back

        //When back clicked change screen to play

        //Go to play with block transition
        application().gotoplayerScreenNoTransition();

        resetFields1();
    }
    if (&src == &restart)
    {
        //restart

        //When restart clicked change screen to game_1

        //Go to game_1 with block transition
        application().gotogame_1ScreenBlockTransition();

        resetFields1();
    }
}

```

Problemi:

Najprej skrivnost 11. zaslona. TouchGFX izgleda generira le zaslone, ki se v aplikaciji uporabljajo (preko interakcij v programu). Zadnji zaslon je bil za gumbе za vse zaslone, ki bodo obstajali v igri, tega zaslona pa uporabnik nikoli ne bo videl.

Naslednja težava je izris križcev in krožcev na zaslon. Vsi elementi so pravzaprav vedno prisotni na zaslonu, le skriti so. Za izris elementov se uporablja npr.

```

middleLeftKrizec.setVisible(true);

middleLeftKrizec.invalidate();

```

To pomeni, da se bo element middleLeftKrizec nastavil na vidnega, funkcija invalidate() pa le posodobi zaslon. Zato sem morala biti pozorna, da se elementov ne prikazuje znotraj podprogramov, sploh kjer je vse odvisno od argumentov funkcij.

Naslednja težava je spreminjanje zaslonov in slik, ko smo že generirali projekt. Načeloma se lahko odpre TouchGFX datoteko po generirani in spremenjeni kodi. V generirani kodi so prav

posebej namenjeni deli za pisanje kode uporabnika, a če ne mislimo več spreminjati zaslonov, se lahko tudi drugo kodo spreminja znotraj CubeIDE in se bo vse izvedlo brez problema. A če pa želimo spremeniti kakšen element ali grafiko, pa se moramo malo bolj potruditi.

Možne nadgradnje:

Vedno je prostor za še več. Lahko bi se projekt nadgradilo z boljšim Minimax algoritmom, saj je ta pomanjkljiv. Lahko bi se dodalo manjši zaslon (npr. LCD zaslon), kjer bi se izpisovale točke posameznega igralca. Lahko bi se pa uporabilo še en večji zaslon in naredilo igro Super TicTacToe, z 3x3 mrežo 3x3 iger. Super zadeva, grozna za implementacijo.